

TRD

Technical Requirements Document (TRD)

AWS Glue ETL Pipeline for Customer Order Processing

•

1. Document Overview

Purpose

This Technical Requirements Document translates the functional requirements into implementation-ready technical specifications for an AWS Glue-based ETL pipeline. It defines the technical architecture, data structures, transformation logic, and operational parameters required to build a production-grade solution that ingests customer and order data, performs data quality operations, implements SCD Type 2 using Apache Hudi, and produces analytical aggregations.

Scope

- In Scope:
 - AWS Glue Spark job implementation using PySpark
 - S3 data ingestion from CSV sources
 - Data quality transformations (null removal, deduplication)
 - AWS Glue Data Catalog table registration and metadata management
 - Apache Hudi table format implementation for SCD Type 2
 - Incremental processing logic based on change detection
 - Aggregation pipeline for customer spending analytics
 - CloudWatch logging and monitoring integration
 - Athena query enablement through catalog registration
- Out of Scope:
 - Glue job scheduling/orchestration (Step Functions, EventBridge)
 - Data quality profiling beyond specified rules
 - Lake Formation fine-grained access control
 - Data lifecycle/archival automation
 - Real-time streaming architecture
 - UI/BI tool integration
 - Automated testing framework

Assumptions

1. Data Format Assumptions:

- CSV files use comma delimiter with header row
- Character encoding is UTF-8
- Date fields in order data are in ISO 8601 format (YYYY-MM-DD) or parseable string format
- Numeric fields (PricePerUnit, Qty) are stored as strings in CSV and require casting to decimal/integer

2. Schema Assumptions:

- CustId: String type, primary key for customer data
- OrderId: String type, primary key for order data
- Name: String type
- EmailId: String type
- Region: String type
- ItemName: String type
- PricePerUnit: Decimal(10,2)
- Qty: Integer
- Date: Date type
- TotalAmount (derived): Decimal(12,2)

3. Infrastructure Assumptions:

- AWS Glue Data Catalog database `gen\_ai\_poc\_databrickscoe` exists
- IAM role attached to Glue job has permissions:
- s3:GetObject on s3://adif-sdlc/sdlc_wizard/
- s3:PutObject, s3:DeleteObject on s3://adif-sdlc/curated/ and s3://adif-sdlc/analytics/
- glue:CreateTable, glue:UpdateTable, glue:GetTable, glue:GetDatabase
- logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents
- Glue job version 4.0 or higher with Hudi 0.12.x support
- Spark 3.3.x runtime environment

4. Processing Assumptions:

- "Null" string literal (case-sensitive) and actual NULL values are the only null representations
- Duplicate detection uses all columns (full row comparison)
- SCD Type 2 change detection compares all customer attributes except CustId
- Initial load creates all records as active (IsActive=true, EndDate=null)
- Incremental processing requires state management of previous customer data
- Timestamps use UTC timezone
- OpTs (operation timestamp) uses job execution start time

5. Operational Assumptions:

- Glue job runs in a single AWS region
- No cross-account S3 access required
- Glue job type: Spark ETL (not Streaming or Python Shell)
- DPU allocation determined at deployment time based on data volume
- Retry logic: 2 retries with exponential backoff
- Job timeout: 2 hours maximum

6. Data Volume Assumptions:

- Customer data: < 10 million records per run
- Order data: < 100 million records per run
- Incremental updates affect < 10% of customer base per run
- No single-file size exceeds 5GB
-

2. Technical Requirements

TR-INGEST-001: Customer Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-001
- Objective:

Read customer CSV files from S3 into a Spark DataFrame with schema enforcement and validation.

- Source Details:
 - Source Type: S3 CSV files
 - Source Location: s3://adif-sdlc/sdlc_wizard/customerdata/
 - Source Format: CSV with header
 - Source Pattern: All files matching .csv in the directory
 - Source System: AWS S3
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: df_customer_raw
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	

	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Output Schema:

Same as Input Schema (pass-through at this stage)

- Processing / Transformation Logic:

1. Initialize Spark session with Glue context
2. Use Glue DynamicFrame.from_options() or spark.read.csv() to read from S3

3. Read parameters:

- format: "csv"
- header: true
- inferSchema: false (explicit schema definition)
- encoding: "UTF-8"
- delimiter: ","

4. Apply explicit schema:

```

```
StructType([
StructField("CustId", StringType(), False),
StructField("Name", StringType(), True),
StructField("EmailId", StringType(), True),
StructField("Region", StringType(), True)
])
```

```

5. Convert to Spark DataFrame for downstream processing

6. Cache DataFrame if used multiple times

- Validation Rules:

- Verify S3 path exists before read operation
- Validate that at least one CSV file is present
- Confirm header row contains expected column names (case-insensitive match)
- Validate CustId is not null in any row (fail job if violated)
- Check for schema mismatch between files if multiple CSVs exist

- Error Handling and Logging:

- Log INFO: "Starting customer data ingestion from s3://adif-sdlc/sdlc_wizard/customerdata/"

- Log INFO: "Customer files found: {file_count}"
- Log INFO: "Customer records loaded: {record_count}"
- Log ERROR: "S3 path not found" → Fail job with exit code 1
- Log ERROR: "No CSV files found in source location" → Fail job with exit code 1
- Log ERROR: "Schema mismatch detected: {details}" → Fail job with exit code 1
- Log ERROR: "CSV parsing failed at file {filename}, line {line_number}" → Fail job with exit code 1
- Emit CloudWatch metric: CustomerRecordsIngested (Count)
- Runtime Parameters:
 - `--customer_source_path` : s3://adif-sdlc/sdlc_wizard/customerdata/ (default)`
 - `--enable_schema_validation` : true (default)`
- Operational Notes:
 - Use Glue bookmark to track processed files if incremental file-based processing is needed
 - Consider S3 Select for large files if filtering at source is beneficial
 - Monitor S3 request costs for high-frequency reads
-

TR-INGEST-002: Order Data Ingestion from S3

- Related Functional Requirement ID(s): FR-INGEST-002
- Objective:

Read order CSV files from S3 into a Spark DataFrame with schema enforcement and validation.

- Source Details:
 - Source Type: S3 CSV files
 - Source Location: s3://adif-sdlc/sdlc_wizard/orderdata/
 - Source Format: CSV with header
 - Source Pattern: All files matching .csv in the directory
 - Source System: AWS S3
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: df_order_raw
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
--	-------------	-----------	------------------------	----------	--

	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	String	Variable	Yes	
	Qty	String	Variable	Yes	
	Date	String	Variable	Yes	
	CustId	String	Variable	Yes	

• Output Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

• Processing / Transformation Logic:

1. Initialize Spark session with Glue context
 2. Use Glue `DynamicFrame.from_options()` or `spark.read.csv()` to read from S3
 3. Read parameters:
 - format: "csv"
 - header: true
 - inferSchema: false (explicit schema definition)
 - encoding: "UTF-8"
 - delimiter: ","
 4. Apply explicit schema for initial read (all String types)
 5. Convert to Spark DataFrame
 6. Cast columns to target types:
 - PricePerUnit: cast to `DecimalType(10,2)`
 - Qty: cast to `IntegerType`
 - Date: cast to `DateType` using `to_date()` function
 7. Handle casting failures by setting failed values to null and logging warnings
- Validation Rules:
- Verify S3 path exists before read operation
 - Validate that at least one CSV file is present

- Confirm header row contains expected column names (case-insensitive match)
- Validate OrderId is not null in any row (fail job if violated)
- Check for schema mismatch between files if multiple CSVs exist
- Log warning for rows where type casting fails
- Error Handling and Logging:
 - Log INFO: "Starting order data ingestion from s3://adif-sdlc/sdlc_wizard/orderdata/"
 - Log INFO: "Order files found: {file_count}"
 - Log INFO: "Order records loaded: {record_count}"
 - Log WARN: "Type casting failed for {failed_count} rows in column {column_name}"
 - Log ERROR: "S3 path not found" → Fail job with exit code 1
 - Log ERROR: "No CSV files found in source location" → Fail job with exit code 1
 - Log ERROR: "Schema mismatch detected: {details}" → Fail job with exit code 1
 - Log ERROR: "CSV parsing failed at file {filename}, line {line_number}" → Fail job with exit code 1
 - Emit CloudWatch metric: OrderRecordsIngested (Count)
 - Emit CloudWatch metric: TypeCastingFailures (Count)
- Runtime Parameters:
 - `--order_source_path`:` s3://adif-sdlc/sdlc_wizard/orderdata/ (default)
 - `--enable_schema_validation`:` true (default)
 - `--date_format`:` "yyyy-MM-dd" (default, configurable for different date formats)
- Operational Notes:
 - Use Glue bookmark to track processed files if incremental file-based processing is needed
 - Type casting failures should be monitored; high failure rates indicate data quality issues
 - Consider S3 Select for large files if filtering at source is beneficial
-

TR-CLEAN-001: Data Quality Processing for Customer Data

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove null values (both string "Null" and actual NULL) and duplicate records from customer dataset.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_customer_raw (from TR-INGEST-001)

- Source System: Spark DataFrame in memory

- Target Details:

- Target Type: In-memory Spark DataFrame

- Target Name: df_customer_clean

- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Output Schema:

Same as Input Schema (cleaned subset)

- Processing / Transformation Logic:

1. Start with df_customer_raw

2. Remove rows where any column contains string literal "Null" (case-sensitive):

- Apply filter condition: NOT (col1 == "Null" OR col2 == "Null" OR ... for all columns)

3. Remove rows where any column contains actual NULL:

- Use dropna() with how="any" parameter

4. Remove duplicate rows:

- Use dropDuplicates() without parameters (considers all columns)

5. Count records at each step for logging

6. Persist cleaned DataFrame if used multiple times

- Validation Rules:

- Log warning if > 50% of records are removed

- Log warning if zero records remain after cleaning

- Validate CustId uniqueness after deduplication

- Error Handling and Logging:

- Log INFO: "Customer records before cleaning: {count_before}"

- Log INFO: "Customer records removed due to 'Null' string: {count_null_string}"

- Log INFO: "Customer records removed due to actual NULL: {count_null_actual}"

- Log INFO: "Customer duplicate records removed: {count_duplicates}"

- Log INFO: "Customer records after cleaning: {count_after}"

- Log WARN: "High data loss detected: {percentage}% of customer records removed"
- Log WARN: "Zero customer records remaining after cleaning"
- Emit CloudWatch metric: CustomerRecordsCleaned (Count)
- Emit CloudWatch metric: CustomerRecordsRemoved (Count)
- Runtime Parameters:
 - `--null_string_literal`: "Null" (default, configurable)
 - `--enable_null_string_removal`: true (default)
 - `--enable_null_value_removal`: true (default)
 - `--enable_deduplication`: true (default)
- Operational Notes:
 - High removal rates indicate upstream data quality issues
 - Consider implementing data quality metrics dashboard
 - Deduplication assumes all columns define uniqueness; adjust if business key differs
-

TR-CLEAN-002: Data Quality Processing for Order Data

- Related Functional Requirement ID(s): FR-CLEAN-001
- Objective:

Remove null values (both string "Null" and actual NULL) and duplicate records from order dataset.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_order_raw (from TR-INGEST-002)
 - Source System: Spark DataFrame in memory
- Target Details:
 - Target Type: In-memory Spark DataFrame
 - Target Name: df_order_clean
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	

	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Output Schema:

Same as Input Schema (cleaned subset)

- Processing / Transformation Logic:

1. Start with df_order_raw (after type casting in TR-INGEST-002)
2. Remove rows where any column contains string literal "Null" (case-sensitive):
 - Apply filter condition for string columns only (OrderId, ItemName, CustId)
3. Remove rows where any column contains actual NULL:
 - Use dropna() with how="any" parameter
4. Remove duplicate rows:
 - Use dropDuplicates() without parameters (considers all columns)
5. Count records at each step for logging
6. Persist cleaned DataFrame if used multiple times

- Validation Rules:

- Log warning if > 50% of records are removed
- Log warning if zero records remain after cleaning
- Validate OrderId uniqueness after deduplication

- Error Handling and Logging:

- Log INFO: "Order records before cleaning: {count_before}"
- Log INFO: "Order records removed due to 'Null' string: {count_null_string}"
- Log INFO: "Order records removed due to actual NULL: {count_null_actual}"
- Log INFO: "Order duplicate records removed: {count_duplicates}"
- Log INFO: "Order records after cleaning: {count_after}"
- Log WARN: "High data loss detected: {percentage}% of order records removed"
- Log WARN: "Zero order records remaining after cleaning"
- Emit CloudWatch metric: OrderRecordsCleaned (Count)
- Emit CloudWatch metric: OrderRecordsRemoved (Count)

- Runtime Parameters:

- `--null_string_literal`: "Null" (default, configurable)
- `--enable_null_string_removal`: true (default)
- `--enable_null_value_removal`: true (default)

- `--enable\_deduplication`: true (default)
- Operational Notes:
 - High removal rates indicate upstream data quality issues
 - Consider implementing data quality metrics dashboard
 - Deduplication assumes all columns define uniqueness; adjust if business key differs
 - Type casting failures from TR-INGEST-002 will result in NULLs that are removed here
-

TR-CATALOG-001: Register Customer Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-001
- Objective:

Write cleaned customer data to Glue Data Catalog as a queryable table accessible via Athena.

- Source Details:
 - Source Type: In-memory Spark DataFrame
 - Source Name: df_customer_clean (from TR-CLEAN-001)
 - Source System: Spark DataFrame in memory
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databricksco
 - Target Table: sdlc_wizard_customer
 - Target Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_customer/
 - Target Format: Parquet
 - Compression: Snappy
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Output Schema:

Same as Input Schema (direct map)

- Processing / Transformation Logic:
 1. Start with df_customer_clean
 2. Write DataFrame to S3 in Parquet format:
 - mode: "overwrite" (replace existing data)
 - format: "parquet"
 - compression: "snappy"
 - path: s3://adif-sdlc/catalog/sdlc_wizard_customer/
 3. Register table in Glue Data Catalog:
 - Use Glue DynamicFrame.write_dynamic_frame.from_catalog() or
 - Use Spark SQL CREATE TABLE statement with LOCATION clause
 4. Update table properties:
 - classification: "parquet"
 - compressionType: "snappy"
 5. Verify table registration via glue.get_table() API call
- Validation Rules:
 - Verify database gen_ai_poc_databrickscoe exists before write
 - Validate write operation completed successfully
 - Confirm table is queryable via Athena test query: SELECT COUNT() FROM table
 - Validate record count matches df_customer_clean count
- Error Handling and Logging:
 - Log INFO: "Writing customer data to s3://adif-sdlc/catalog/sdlc_wizard_customer/"
 - Log INFO: "Customer records written: {count}"
 - Log INFO: "Registering table gen_ai_poc_databrickscoe.sdlc_wizard_customer"
 - Log INFO: "Table registration successful"
 - Log ERROR: "Database gen_ai_poc_databrickscoe not found" → Fail job with exit code 1
 - Log ERROR: "S3 write failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "Table registration failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "Athena validation query failed" → Log warning but continue
 - Emit CloudWatch metric: CustomerTableRecordsWritten (Count)
- Runtime Parameters:
 - `--customer_catalog_path``: s3://adif-sdlc/catalog/sdlc_wizard_customer/ (default)
 - `--customer_table_name``: sdlc_wizard_customer (default)
 - `--catalog_database``: gen_ai_poc_databrickscoe (default)
 - `--write_mode``: overwrite (default, options: overwrite, append)

- `--enable\_athena\_validation`: true (default)
- Operational Notes:
- Overwrite mode replaces entire table; consider partitioning for incremental updates
- Parquet format provides efficient columnar storage and compression
- Table is immediately queryable via Athena after registration
- Monitor S3 storage costs for catalog tables
-

TR-CATALOG-002: Register Order Table in Glue Data Catalog

- Related Functional Requirement ID(s): FR-CATALOG-002
- Objective:

Write cleaned order data to Glue Data Catalog as a queryable table accessible via Athena.

- Source Details:
- Source Type: In-memory Spark DataFrame
- Source Name: df_order_clean (from TR-CLEAN-002)
- Source System: Spark DataFrame in memory
- Target Details:
- Target Type: AWS Glue Data Catalog Table
- Target Database: gen_ai_poc_databrickscoe
- Target Table: sdlc_wizard_order
- Target Storage Location: s3://adif-sdlc/catalog/sdlc_wizard_order/
- Target Format: Parquet
- Compression: Snappy
- Input Schema:

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Output Schema:

Same as Input Schema (direct map)

- Processing / Transformation Logic:

1. Start with df_order_clean

2. Write DataFrame to S3 in Parquet format:

- mode: "overwrite" (replace existing data)

- format: "parquet"

- compression: "snappy"

- path: s3://adif-sdlc/catalog/sdlc_wizard_order/

3. Register table in Glue Data Catalog:

- Use Glue DynamicFrame.write_dynamic_frame.from_catalog() or

- Use Spark SQL CREATE TABLE statement with LOCATION clause

4. Update table properties:

- classification: "parquet"

- compressionType: "snappy"

5. Verify table registration via glue.get_table() API call

- Validation Rules:

- Verify database gen_ai_poc_databrickscoe exists before write

- Validate write operation completed successfully

- Confirm table is queryable via Athena test query: SELECT COUNT() FROM table

- Validate record count matches df_order_clean count

- Error Handling and Logging:

- Log INFO: "Writing order data to s3://adif-sdlc/catalog/sdlc_wizard_order/"

- Log INFO: "Order records written: {count}"

- Log INFO: "Registering table gen_ai_poc_databrickscoe.sdlc_wizard_order"

- Log INFO: "Table registration successful"

- Log ERROR: "Database gen_ai_poc_databrickscoe not found" → Fail job with exit code 1

- Log ERROR: "S3 write failed: {error_details}" → Fail job with exit code 1

- Log ERROR: "Table registration failed: {error_details}" → Fail job with exit code 1

- Log ERROR: "Athena validation query failed" → Log warning but continue

- Emit CloudWatch metric: OrderTableRecordsWritten (Count)

- Runtime Parameters:

- `--order\_catalog\_path`: s3://adif-sdlc/catalog/sdlc_wizard_order/ (default)

- `--order\_table\_name`: sdlc_wizard_order (default)

- `--catalog_database` : gen_ai_poc_databrickscoe (default)`
- `--write_mode` : overwrite (default, options: overwrite, append)`
- `--enable_athena_validation` : true (default)`
- Operational Notes:
- Overwrite mode replaces entire table; consider partitioning for incremental updates
- Parquet format provides efficient columnar storage and compression
- Table is immediately queryable via Athena after registration
- Monitor S3 storage costs for catalog tables
-

TR-SCD2-001: Implement SCD Type 2 for Order Summary Using Hudi

- Related Functional Requirement ID(s): FR-SCD2-001
- Objective:

Join customer and order data, implement SCD Type 2 logic to track historical changes, and write results to Hudi table format.

- Source Details:
- Source Type 1: In-memory Spark DataFrame
- Source Name 1: df_customer_clean (from TR-CLEAN-001)
- Source System 1: Spark DataFrame in memory
- Source Type 2: In-memory Spark DataFrame
- Source Name 2: df_order_clean (from TR-CLEAN-002)
- Source System 2: Spark DataFrame in memory
- Source Type 3: Hudi Table (for incremental updates)
- Source Name 3: ordersummary (existing Hudi table)
- Source Location 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
- Target Type: Apache Hudi Table (Copy-on-Write)
- Target Database: gen_ai_poc_databrickscoe
- Target Table: ordersummary
- Target Storage Location: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Format: Hudi (Copy-on-Write)
- Hudi Table Type: COPY_ON_WRITE
- Record Key: OrderId

- Precombine Key: OpTs
- Partition Path: (none - unpartitioned for simplicity; can be added if needed)
- Input Schema (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Input Schema (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Output Schema (ordersummary):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Derivation	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map from df_order_clean.OrderId	
	ItemName	String	Variable	Yes	Direct map from df_order_clean.ItemName	
	PricePerUnit	Decimal	10,2	Yes	Direct map from df_order_clean.PricePerUnit	
	Qty	Integer	-	Yes	Direct map from df_order_clean.Qty	
	Date	Date	-	Yes	Direct map from df_order_clean.Date	
	CustId	String	Variable	Yes	Direct map from df_order_clean.CustId	
	Name	String	Variable	Yes	Direct map from df_customer_clean.Name	
	EmailId	String	Variable	Yes	Direct map from df_customer_clean.EmailId	
	Region	String	Variable	Yes	Direct map from df_customer_clean.Region	
	IsActive	Boolean	-	No	SCD Type 2 flag: true for current, false for historical	
	StartDate	Timestamp	-	No	SCD Type 2: record effective start timestamp	
	EndDate	Timestamp	-	Yes	SCD Type 2: record effective end timestamp (null for active)	
	OpTs	Timestamp	-	No	Operation timestamp (job execution time)	

- Processing / Transformation Logic:

- Initial Load (First Run):

1. Join df_customer_clean and df_order_clean on CustId (inner join)

2. Add SCD Type 2 columns:

- IsActive = true
- StartDate = current_timestamp()
- EndDate = null
- OpTs = current_timestamp()

3. Write to Hudi table with operation type: "insert"

- Incremental Update (Subsequent Runs):

1. Read existing ordersummary Hudi table into df_ordersummary_existing

2. Join df_customer_clean and df_order_clean on CustId (inner join) → df_order_customer_joined

3. Identify new orders (OrderId not in df_ordersummary_existing):

- Add SCD Type 2 columns as in initial load
- Stage for insert operation

4. Identify existing orders with customer changes:

- Join df_order_customer_joined with df_ordersummary_existing on OrderId

- Compare customer attributes (Name, EmailId, Region)

- If any customer attribute differs:

- Close existing active record:

- Set IsActive = false
- Set EndDate = current_timestamp()
- Set OpTs = current_timestamp()

- Create new active record:

- Copy all order attributes
- Use updated customer attributes
- Set IsActive = true
- Set StartDate = current_timestamp()
- Set EndDate = null
- Set OpTs = current_timestamp()

5. Write updates to Hudi table:

- Use Hudi upsert operation for changed records
- Use Hudi insert operation for new records

- Hudi Configuration:

...

```

hoodi_options = {
'hoodie.table.name': 'ordersummary',
'hoodie.datasource.write.recordkey.field': 'OrderId',
'hoodie.datasource.write.precombine.field': 'OpTs',
'hoodie.datasource.write.table.type': 'COPY_ON_WRITE',
'hoodie.datasource.write.operation': 'upsert', # or 'insert' for initial load
'hoodie.datasource.hive_sync.enable': 'true',
'hoodie.datasource.hive_sync.database': 'gen_ai_poc_databrickscoe',
'hoodie.datasource.hive_sync.table': 'ordersummary',
'hoodie.datasource.hive_sync.mode': 'hms',
'hoodie.datasource.hive_sync.partition_extractor_class': 'org.apache.hudi.hive.NonPartitionedExtractor'
}
...

```

- Validation Rules:
 - Validate no duplicate active records exist for same OrderId after write
 - Validate all active records have EndDate = null
 - Validate all historical records have EndDate populated
 - Validate StartDate < EndDate for all historical records
 - Validate join produces expected record count (log warning if zero)
- Error Handling and Logging:
 - Log INFO: "Starting SCD Type 2 processing for ordersummary"
 - Log INFO: "Joined customer-order records: {count}"
 - Log INFO: "New orders identified: {count}"
 - Log INFO: "Orders with customer changes: {count}"
 - Log INFO: "Records to insert: {count}"
 - Log INFO: "Records to update: {count}"
 - Log INFO: "Writing to Hudi table at s3://adif-sdlc/curated/sdlc_wizard/ordersummary/"
 - Log INFO: "Hudi write completed successfully"
 - Log ERROR: "Join produced zero records" → Log warning and create empty table
 - Log ERROR: "Hudi write failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "Duplicate active records detected for OrderId: {order_ids}" → Fail job with exit code 1
 - Log ERROR: "Hive sync failed: {error_details}" → Log warning but continue
 - Emit CloudWatch metric: OrderSummaryRecordsInserted (Count)

- Emit CloudWatch metric: OrderSummaryRecordsUpdated (Count)
- Emit CloudWatch metric: OrderSummaryActiveRecords (Count)
- Runtime Parameters:
 - `--ordersummary_path``: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (default)
 - `--ordersummary_table_name``: ordersummary (default)
 - `--catalog_database``: gen_ai_poc_databrickscoe (default)
 - `--hudi_table_type``: COPY_ON_WRITE (default, options: COPY_ON_WRITE, MERGE_ON_READ)
 - `--enable_hive_sync``: true (default)
 - `--is_initial_load``: false (default, set to true for first run)
- Operational Notes:
 - Initial load should set `--is_initial_load=true`` to skip change detection
 - Hudi COPY_ON_WRITE provides snapshot isolation for queries
 - Hudi Hive sync enables Athena querying with time-travel capability
 - Monitor Hudi compaction and cleaning operations
 - Consider partitioning by Date if data volume grows significantly
 - SCD Type 2 logic assumes customer changes are detected by comparing all attributes except CustId
-

TR-INCR-001: Incremental Processing Based on Customer Updates

- Related Functional Requirement ID(s): FR-INCR-001
- Objective:

Detect changes in customer data and trigger incremental SCD Type 2 updates to ordersummary table for affected records only.

- Source Details:
 - Source Type 1: In-memory Spark DataFrame
 - Source Name 1: df_customer_clean (current run, from TR-CLEAN-001)
 - Source System 1: Spark DataFrame in memory
 - Source Type 2: S3 Parquet (previous run customer data)
 - Source Name 2: customer_previous
 - Source Location 2: s3://adif-sdlc/state/customer_previous/
 - Source System 2: AWS S3
 - Source Type 3: Hudi Table

- Source Name 3: ordersummary (existing Hudi table)
- Source Location 3: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Details:
- Target Type 1: Hudi Table (updated ordersummary)
- Target Location 1: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Target Type 2: S3 Parquet (state management for next run)
- Target Location 2: s3://adif-sdlc/state/customer_previous/
- Input Schema (df_customer_clean - current):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Input Schema (customer_previous):

Same as df_customer_clean

- Output Schema:

No direct output schema; triggers TR-SCD2-001 processing for affected records

- Processing / Transformation Logic:

- Change Detection:

1. Read previous customer state from s3://adif-sdlc/state/customer_previous/

- If path does not exist (first run), treat all customers as new

2. Compare df_customer_clean (current) with customer_previous:

- Identify new customers: CustId in current but not in previous
- Identify changed customers: CustId in both, but any attribute (Name, EmailId, Region) differs
- Use full outer join on CustId with comparison logic

3. Create df_customer_changes containing only new and changed customer records

- Incremental Update Trigger:

4. If df_customer_changes is empty:

- Log INFO: "No customer changes detected, skipping ordersummary update"
- Skip TR-SCD2-001 processing
- Proceed to state update

5. If `df_customer_changes` is not empty:

- Filter `df_order_clean` to include only orders for changed customers (`CustId` in `df_customer_changes`)
- Pass filtered customer and order DataFrames to TR-SCD2-001 for incremental update
- TR-SCD2-001 will handle SCD Type 2 logic for affected orders

• State Management:

6. After processing, write `df_customer_clean` to `s3://adif-sdlc/state/customer_previous/`

- mode: "overwrite"
- format: "parquet"
- This becomes the baseline for next run's change detection
- Validation Rules:
 - Validate state path is accessible for read/write
 - Validate change detection logic correctly identifies new vs. changed customers
 - Validate filtered order dataset contains only orders for changed customers
- Error Handling and Logging:
 - Log INFO: "Reading previous customer state from `s3://adif-sdlc/state/customer_previous/`"
 - Log INFO: "Previous customer state not found, treating as initial load"
 - Log INFO: "Current customer records: {count_current}"
 - Log INFO: "Previous customer records: {count_previous}"
 - Log INFO: "New customers detected: {count_new}"
 - Log INFO: "Changed customers detected: {count_changed}"
 - Log INFO: "Total customer changes: {count_total_changes}"
 - Log INFO: "No customer changes detected, skipping ordersummary update"
 - Log INFO: "Filtering orders for changed customers"
 - Log INFO: "Orders affected by customer changes: {count_orders}"
 - Log INFO: "Triggering incremental SCD Type 2 update"
 - Log INFO: "Writing current customer state to `s3://adif-sdlc/state/customer_previous/`"
 - Log ERROR: "State path read failed: {error_details}" → Treat as initial load
 - Log ERROR: "State path write failed: {error_details}" → Fail job with exit code 1
 - Emit CloudWatch metric: CustomerChangesDetected (Count)
 - Emit CloudWatch metric: OrdersAffectedByCustomerChanges (Count)
- Runtime Parameters:
 - `--customer_state_path``: `s3://adif-sdlc/state/customer_previous/` (default)

- `--enable_incremental_processing` : true (default)`
- `--force_full_refresh` : false (default, set to true to skip change detection)`
- Operational Notes:
 - First run will treat all customers as new (no previous state)
 - State management requires persistent S3 storage between runs
 - Force full refresh option bypasses change detection for troubleshooting
 - Monitor state file size; consider compaction if it grows large
 - Change detection compares all customer attributes; adjust if business key differs
 - Incremental processing significantly reduces processing time for large datasets
-

TR-AGG-001: Generate Customer Aggregate Spend Table

- Related Functional Requirement ID(s): FR-AGG-001
- Objective:

Join customer and order data, calculate total spending per customer per date, and write aggregated results to S3 and Glue Data Catalog.

- Source Details:
 - Source Type 1: In-memory Spark DataFrame
 - Source Name 1: df_customer_clean (from TR-CLEAN-001)
 - Source System 1: Spark DataFrame in memory
 - Source Type 2: In-memory Spark DataFrame
 - Source Name 2: df_order_clean (from TR-CLEAN-002)
 - Source System 2: Spark DataFrame in memory
- Target Details:
 - Target Type: AWS Glue Data Catalog Table
 - Target Database: gen_ai_poc_databricksco
 - Target Table: customeraggregatespend
 - Target Storage Location: s3://adif-sdlc/analytics/customeraggregatespend/
 - Target Format: Parquet
 - Compression: Snappy
- Input Schema (df_customer_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Input Schema (df_order_clean):

	Column Name	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Output Schema (customeraggregatespend):

	Column Name	Data Type	Length/Precision/Scale	Nullable	Derivation	
	-----	-----	-----	-----	-----	
	Name	String	Variable	Yes	Direct map from df_customer_clean.Name	
	TotalAmount	Decimal	12,2	Yes	SUM(PricePerUnit * Qty) grouped by Name, Date	
	Date	Date	-	Yes	Direct map from df_order_clean.Date	

- Processing / Transformation Logic:

1. Join df_customer_clean and df_order_clean on CustId (inner join)

2. Calculate TotalAmount for each order:

- TotalAmount = PricePerUnit Qty

- Use Spark SQL expression: `col("PricePerUnit") col("Qty")`

3. Group by Name and Date

4. Aggregate: SUM(TotalAmount) as TotalAmount

5. Select final columns: Name, TotalAmount, Date

6. Handle null values:

- Exclude rows where Name is null
- Exclude rows where Date is null
- Exclude rows where PricePerUnit or Qty is null (these will produce null TotalAmount)

7. Write to S3 in Parquet format:

- mode: "overwrite"
- format: "parquet"
- compression: "snappy"

- path: s3://adif-sdlc/analytics/customeraggregatespend/

8. Register table in Glue Data Catalog:

- database: gen_ai_poc_databrickscoe
- table: customeraggregatespend

• Validation Rules:

- Validate join produces records (log warning if zero)
- Validate TotalAmount is non-negative (log warning for negative values)
- Validate aggregation produces expected record count
- Validate table is queryable via Athena after registration

• Error Handling and Logging:

- Log INFO: "Starting customer aggregate spend calculation"
 - Log INFO: "Joining customer and order data"
 - Log INFO: "Joined records: {count}"
 - Log INFO: "Calculating TotalAmount per order"
 - Log INFO: "Grouping by Name and Date"
 - Log INFO: "Aggregated records: {count}"
 - Log INFO: "Writing to s3://adif-sdlc/analytics/customeraggregatespend/"
 - Log INFO: "Records written: {count}"
 - Log INFO: "Registering table gen_ai_poc_databrickscoe.customeraggregatespend"
 - Log INFO: "Table registration successful"
 - Log WARN: "Join produced zero records"
 - Log WARN: "Negative TotalAmount detected for {count} records"
 - Log WARN: "Null values excluded: Name={count_name}, Date={count_date}, TotalAmount={count_amount}"
 - Log ERROR: "Join failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "Aggregation failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "S3 write failed: {error_details}" → Fail job with exit code 1
 - Log ERROR: "Table registration failed: {error_details}" → Fail job with exit code 1
 - Emit CloudWatch metric: CustomerAggregateRecordsWritten (Count)
 - Emit CloudWatch metric: TotalSpendCalculated (Sum)
- #### • Runtime Parameters:
- `--customeraggregatespend_path` : s3://adif-sdlc/analytics/customeraggregatespend/ (default)`
 - `--customeraggregatespend_table_name` : customeraggregatespend (default)`
 - `--catalog_database` : gen_ai_poc_databrickscoe (default)`

- `--write_mode`: overwrite (default)
- `--enable_athena_validation`: true (default)
- Operational Notes:
 - Aggregation is performed at customer name and date level
 - TotalAmount calculation assumes PricePerUnit and Qty are valid numeric values
 - Null handling excludes incomplete records from aggregation
 - Overwrite mode replaces entire table; consider incremental aggregation for large datasets
 - Monitor for negative TotalAmount values indicating data quality issues
 - Table is immediately queryable via Athena after registration
-

3. Runtime Strategy Notes

Authoring Language

- Primary Language: PySpark (Python 3.9+)
- Rationale: Native Glue support, rich Spark API, Hudi integration, readable syntax
- Alternative: Scala (if performance optimization is critical)

Execution Strategy

- Job Type: AWS Glue Spark ETL Job
- Glue Version: 4.0 (Spark 3.3.0, Python 3.10)
- Worker Type: G.1X (4 vCPU, 16 GB memory) or G.2X for larger datasets
- Number of Workers: Start with 5, scale based on data volume
- Max Concurrent Runs: 1 (prevent concurrent updates to Hudi table)
- Job Timeout: 2 hours (7200 seconds)
- Max Retries: 2
- Delay Between Retries: 300 seconds (5 minutes)
- Execution Mode: Standard (not Flex for predictable performance)

Glue Job Type Expectations

- Job Type: Spark ETL (not Streaming or Python Shell)
- Script Location: `s3:///scripts/customer_order_etl.py`
- Temporary Directory: `s3:///temp/`
- Spark UI Logs: `s3:///sparkui-logs/`
- CloudWatch Logs: Enabled (default)
- Job Bookmark: Disabled (using custom state management for incremental processing)

- Continuous Logging: Enabled (real-time CloudWatch logs)
- Metrics: Enabled (CloudWatch metrics for job execution)

Dependencies

- Hudi Libraries: Included in Glue 4.0 (Hudi 0.12.x)
- Additional JARs: None required (Hudi bundled with Glue)
- Python Libraries: None beyond Glue defaults
- Glue Connections: None required (S3 access via IAM role)

IAM Role Requirements

- Service: glue.amazonaws.com
- Policies:
 - AWSGlueServiceRole (managed policy)
 - Custom policy for S3 access:
 - s3:GetObject on s3://adif-sdlc/sdlc_wizard/
 - s3:PutObject, s3:DeleteObject on s3://adif-sdlc/curated/, s3://adif-sdlc/analytics/, s3://adif-sdlc/catalog/, s3://adif-sdlc/state/
 - Custom policy for Glue Catalog:
 - glue:GetDatabase, glue:GetTable, glue:CreateTable, glue:UpdateTable on database gen_ai_poc_databrickscoe
- CloudWatch Logs:
 - logs:CreateLogGroup, logs:CreateLogStream, logs:PutLogEvents
-

4. Data Design Details

Source Paths / Source Systems

- Customer Data Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV files)
- Order Data Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV files)
- Previous Customer State (for incremental processing): s3://adif-sdlc/state/customer_previous/ (Parquet)
- Existing Order Summary (for SCD Type 2): s3://adif-sdlc/curated/sdlc_wizard/ordersummary/ (Hudi)

Target Paths / Target Tables

- Customer Catalog Table:
 - Path: s3://adif-sdlc/catalog/sdlc_wizard_customer/
 - Database: gen_ai_poc_databrickscoe
 - Table: sdlc_wizard_customer

- Format: Parquet
- Order Catalog Table:
- Path: s3://adif-sdlc/catalog/sdlc_wizard_order/
- Database: gen_ai_poc_databricksco
- Table: sdlc_wizard_order
- Format: Parquet
- Order Summary (SCD Type 2):
- Path: s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
- Database: gen_ai_poc_databricksco
- Table: ordersummary
- Format: Hudi (COPY_ON_WRITE)
- Customer Aggregate Spend:
- Path: s3://adif-sdlc/analytics/customeraggregatespend/
- Database: gen_ai_poc_databricksco
- Table: customeraggregatespend
- Format: Parquet
- Customer State (for incremental processing):
- Path: s3://adif-sdlc/state/customer_previous/
- Format: Parquet (not cataloged)

Catalog Registration Expectations

- All target tables (except state management) must be registered in Glue Data Catalog
- Catalog registration enables Athena querying
- Hudi tables use Hive sync for automatic catalog updates
- Parquet tables use Glue API or Spark SQL for catalog registration
- Table properties should include:
- classification (parquet or hudi)
- compressionType (snappy)
- location (S3 path)

Partitioning Expectations

- Initial Implementation: No partitioning (unpartitioned tables)
- Future Consideration: Partition ordersummary and customeraggregatespend by Date if data volume exceeds 100M records
- Partitioning Strategy (if implemented):
- ordersummary: Partition by Date (daily partitions)

- customeraggregatespend: Partition by Date (daily partitions)
- Partition format: Date=YYYY-MM-DD

File Formats / Table Formats

- Source Files: CSV (comma-delimited, UTF-8, with header)
- Catalog Tables: Parquet (columnar, Snappy compression)
- SCD Type 2 Table: Hudi COPY_ON_WRITE (Parquet base format)
- State Management: Parquet (Snappy compression)

Update / Overwrite / Append Behavior

- Customer Catalog Table (sdlc_wizard_customer): Overwrite mode (full refresh each run)
- Order Catalog Table (sdlc_wizard_order): Overwrite mode (full refresh each run)
- Order Summary (ordersummary): Upsert mode via Hudi (incremental SCD Type 2 updates)
- Customer Aggregate Spend (customeraggregatespend): Overwrite mode (full refresh each run)
- Customer State (customer_previous): Overwrite mode (replace with current state)
-

5. Processing Details

Data Quality Rules

1. Null Removal:

- Remove rows where any column contains string literal "Null" (case-sensitive)
- Remove rows where any column contains actual NULL value
- Apply to both customer and order datasets independently

2. Deduplication:

- Remove duplicate rows based on all columns (full row comparison)
- Apply to both customer and order datasets independently

3. Type Validation:

- Validate PricePerUnit and Qty are numeric after casting
- Validate Date is parseable to Date type
- Log warnings for type casting failures

4. Business Rule Validation:

- CustId must not be null (enforced by schema)
- OrderId must not be null (enforced by schema)
- TotalAmount should be non-negative (log warning if negative)

Deduplication Rules

- Method: Spark dropDuplicates() without parameters
- Scope: All columns in each dataset
- Behavior: Keeps first occurrence, removes subsequent duplicates
- Logging: Log count of duplicates removed per dataset

Join Rules

1. Customer-Order Join (for SCD Type 2 and Aggregation):
 - Join Type: Inner join
 - Join Key: CustId
 - Behavior: Excludes orders without matching customer and customers without orders
 - Null Handling: Rows with null CustId are excluded by data quality rules
2. Customer Change Detection Join (for Incremental Processing):
 - Join Type: Full outer join
 - Join Key: CustId
 - Behavior: Identifies new, changed, and unchanged customers
 - Comparison: All attributes except CustId

Aggregation Rules

1. Customer Aggregate Spend:
 - Group By: Name, Date
 - Aggregation: SUM(PricePerUnit Qty) as TotalAmount
 - Null Handling: Exclude rows where Name, Date, PricePerUnit, or Qty is null
 - Output: One row per customer name per date with total spending

SCD / History Tracking Rules

1. SCD Type 2 Implementation:
 - Business Key: OrderId (uniquely identifies each order)
 - Change Detection Attributes: Name, EmailId, Region (customer attributes)
 - SCD Metadata Columns:
 - IsActive (Boolean): true for current record, false for historical
 - StartDate (Timestamp): Effective start date of the record
 - EndDate (Timestamp): Effective end date (null for active records)
 - OpTs (Timestamp): Operation timestamp (job execution time)
2. Change Detection Logic:
 - Compare current customer attributes with previous state
 - If any attribute differs, close existing active record and insert new active record
 - If no change, leave existing record unchanged

3. Initial Load:

- All records inserted with IsActive=true, EndDate=null

4. Incremental Update:

- New orders: Insert with IsActive=true, EndDate=null
- Changed customer attributes: Close old record (IsActive=false, EndDate=current_timestamp), insert new record (IsActive=true, EndDate=null)

5. Hudi Upsert Behavior:

- Record Key: OrderId
- Precombine Key: OpTs (latest timestamp wins)
- Operation: Upsert (insert new, update existing)

Null Handling

1. Source Data:

- String literal "Null" treated as null and removed
- Actual NULL values removed

2. Type Casting:

- Failed casts result in NULL values
- NULL values removed by data quality rules

3. Aggregation:

- Rows with NULL in grouping columns (Name, Date) excluded
- Rows with NULL in aggregation columns (PricePerUnit, Qty) excluded

4. SCD Type 2:

- EndDate is NULL for active records (intentional)
- All other columns should not be NULL after data quality processing

Type Casting Expectations

1. Order Data Type Casting:

- PricePerUnit: String → Decimal(10,2)
- Qty: String → Integer
- Date: String → Date (using to_date() with format "yyyy-MM-dd")

2. Casting Failure Handling:

- Failed casts result in NULL values
- NULL values logged as warnings
- NULL values removed by data quality rules

3. Timestamp Generation:

- StartDate, EndDate, OpTs: Generated using current_timestamp() function

- Timezone: UTC (Spark default)

•

6. Traceability Matrix

Business Source	Technical Source
-----	-----
s3://adif-sdlc/sdlc_wizard/customerdata/	s3://adif-sdlc/sdlc_wizard/customerdata/
s3://adif-sdlc/sdlc_wizard/orderdata/	s3://adif-sdlc/sdlc_wizard/orderdata/
customer_raw, df_order_raw	df_customer_raw, df_order_raw
customer_clean	df_customer_clean
order_clean	df_order_clean
customer_clean, df_order_clean, ordersummary (existing)	df_customer_clean, df_order_clean, s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
customer_clean, customer_previous, ordersummary	df_customer_clean, s3://adif-sdlc/state/customer_previous/, s3://adif-sdlc/curated/sdlc_wizard/ordersummary/
customer_clean, df_order_clean	df_customer_clean, df_order_clean

- Source Fidelity Verification:
- All sources in TRD match exactly with sources specified in FRD
- No intermediate sources or staging tables introduced
- State management (customer_previous) is explicitly required for incremental processing per FR-INCR-001
- Hudi table (ordersummary) is both source and target for SCD Type 2 updates per FR-SCD2-001
-

7. ETL Mapping Requirements

TR-INGEST-001: Customer Data Ingestion

- Source: s3://adif-sdlc/sdlc_wizard/customerdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Target: df_customer_raw (Spark DataFrame)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	

	CustId	String	Variable	No	Direct map from source CustId	
	Name	String	Variable	Yes	Direct map from source Name	
	EmailId	String	Variable	Yes	Direct map from source EmailId	
	Region	String	Variable	Yes	Direct map from source Region	

•

TR-INGEST-002: Order Data Ingestion

- Source: s3://adif-sdlc/sdlc_wizard/orderdata/ (CSV)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	String	Variable	Yes	
	Qty	String	Variable	Yes	
	Date	String	Variable	Yes	
	CustId	String	Variable	Yes	

- Target: df_order_raw (Spark DataFrame - after type casting)

Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
-----	-----	-----	-----	-----	
OrderId	String	Variable	No	Direct map from source OrderId	
ItemName	String	Variable	Yes	Direct map from source ItemName	
PricePerUnit	Decimal	10,2	Yes	Cast from source PricePerUnit (String → Decimal)	
Qty	Integer	-	Yes	Cast from source Qty (String → Integer)	
Date	Date	-	Yes	Cast from source Date (String → Date)	
CustId	String	Variable	Yes	Direct map from source CustId	

•

TR-CLEAN-001: Customer Data Cleaning

- Source: df_customer_raw (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Target: df_customer_clean (Spark DataFrame)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map (after null and duplicate removal)	
	Name	String	Variable	Yes	Direct map (after null and duplicate removal)	
	EmailId	String	Variable	Yes	Direct map (after null and duplicate removal)	
	Region	String	Variable	Yes	Direct map (after null and duplicate removal)	

•

TR-CLEAN-002: Order Data Cleaning

- Source: df_order_raw (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Target: df_order_clean (Spark DataFrame)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map (after null and duplicate removal)	
	ItemName	String	Variable	Yes	Direct map (after null and duplicate removal)	
	PricePerUnit	Decimal	10,2	Yes	Direct map (after null and duplicate removal)	
	Qty	Integer	-	Yes	Direct map (after null and duplicate removal)	
	Date	Date	-	Yes	Direct map (after null and duplicate removal)	
	CustId	String	Variable	Yes	Direct map (after null and duplicate removal)	

•

TR-CATALOG-001: Customer Catalog Table

- Source: df_customer_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	

	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Target: gen_ai_poc_databrickscoe.sdlic_wizard_customer (Glue Catalog Table, Parquet)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	CustId	String	Variable	No	Direct map from source CustId	
	Name	String	Variable	Yes	Direct map from source Name	
	EmailId	String	Variable	Yes	Direct map from source EmailId	
	Region	String	Variable	Yes	Direct map from source Region	

•

TR-CATALOG-002: Order Catalog Table

- Source: df_order_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Target: gen_ai_poc_databrickscoe.sdlic_wizard_order (Glue Catalog Table, Parquet)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map from source OrderId	
	ItemName	String	Variable	Yes	Direct map from source ItemName	
	PricePerUnit	Decimal	10,2	Yes	Direct map from source PricePerUnit	
	Qty	Integer	-	Yes	Direct map from source Qty	
	Date	Date	-	Yes	Direct map from source Date	
	CustId	String	Variable	Yes	Direct map from source CustId	

•

TR-SCD2-001: Order Summary with SCD Type 2

- Source 1: df_customer_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Source 2: df_order_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Target: gen_ai_poc_databrickscoe.ordersummary (Hudi Table)

	Target Column	Data Type	Length/Precision/Scale	Nullable	Mapping	
	-----	-----	-----	-----	-----	
	OrderId	String	Variable	No	Direct map from df_order_clean.OrderId	
	ItemName	String	Variable	Yes	Direct map from df_order_clean.ItemName	
	PricePerUnit	Decimal	10,2	Yes	Direct map from df_order_clean.PricePerUnit	
	Qty	Integer	-	Yes	Direct map from df_order_clean.Qty	
	Date	Date	-	Yes	Direct map from df_order_clean.Date	
	CustId	String	Variable	Yes	Direct map from df_order_clean.CustId	
	Name	String	Variable	Yes	Direct map from df_customer_clean.Name (via join on CustId)	
	EmailId	String	Variable	Yes	Direct map from df_customer_clean.EmailId (via join on CustId)	
	Region	String	Variable	Yes	Direct map from df_customer_clean.Region (via join on CustId)	
	IsActive	Boolean	-	No	Derived: true for active records, false for historical	
	StartDate	Timestamp	-	No	Derived: current_timestamp() at record creation	
	EndDate	Timestamp	-	Yes	Derived: null for active, current_timestamp() when closed	
	OpTs	Timestamp	-	No	Derived: current_timestamp() at operation time	

•

TR-AGG-001: Customer Aggregate Spend

- Source 1: df_customer_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	CustId	String	Variable	No	
	Name	String	Variable	Yes	
	EmailId	String	Variable	Yes	
	Region	String	Variable	Yes	

- Source 2: df_order_clean (Spark DataFrame)

	Source Column	Data Type	Length/Precision/Scale	Nullable	
	-----	-----	-----	-----	
	OrderId	String	Variable	No	
	ItemName	String	Variable	Yes	
	PricePerUnit	Decimal	10,2	Yes	
	Qty	Integer	-	Yes	
	Date	Date	-	Yes	
	CustId	String	Variable	Yes	

- Target: gen_ai_poc_databrickscoe.customeragggregatespend (Glue Catalog Table, Parquet)

Source Column	Data Type	Length/Precision/Scale	Nullable	Mapping
-----	-----	-----	-----	-----
	String	Variable	Yes	Direct map from df_customer_clean.Name (via join on CustId)
Amount	Decimal	12,2	Yes	Derived: SUM(df_order_clean.PricePerUnit * df_order_clean.Qty) grouped by Name
	Date	-	Yes	Direct map from df_order_clean.Date

•

Assumptions

- Schema Assumptions:
 - Not enough information available in the FRD to derive exact string length constraints for variable-length string columns (CustId, Name, EmailId, Region, OrderId, ItemName). Implementation will use Spark StringType without explicit length limits.
 - Date format in source CSV files is assumed to be "yyyy-MM-dd" (ISO 8601). If different, the `--date\_format` runtime parameter should be adjusted.
 - Decimal precision for PricePerUnit (10,2) and TotalAmount (12,2) are assumed based on typical financial data requirements. FRD does not specify exact precision/scale.
- Operational Assumptions:

- Glue job execution frequency (daily, hourly, etc.) is not specified in FRD and should be determined based on business requirements.
- Incremental processing trigger mechanism (scheduled, event-driven) is not specified in FRD.
- Data retention policies for source files, catalog tables, and Hudi table are not specified in FRD.
- Disaster recovery and backup strategies are not specified in FRD.
- Performance Assumptions:
 - DPU allocation and worker count should be tuned based on actual data volumes during implementation.
 - Partitioning strategy may be required for tables exceeding 100M records (not specified in FRD).
-
- Document Control:
 - This TRD is derived exclusively from the provided FRD
 - All assumptions are clearly documented
 - All requirements are traceable to FRD requirements
 - Source fidelity is maintained throughout (no intermediate sources introduced)
 - Implementation-ready specifications provided for code generation phase